

Enterprise Database Infrastructure Administration Lab

Design, Deployment, Security Hardening, Backup, Replication and
Monitoring of PostgreSQL Infrastructure on Linux

David Gottlieb SITTI
<https://www.linkedin.com/in/davidgottliebsitti/>

SUMMARY

INCIDENT SIMULATION & OPERATIONAL TROUBLESHOOTING	2
---	---

INCIDENT SIMULATION & OPERATIONAL TROUBLESHOOTING

I. Objective

This deliverable simulates real-world incidents that Database Administrators, Systems Administrators, DevOps Engineers, and Infrastructure Engineers face in production environments.

The goal is not to break things randomly.

The goal is to learn :

- Incident Detection
- Incident Investigation
- Incident Containment
- Incident Resolution
- Root Cause Analysis (RCA)
- Post-Incident Documentation

At the end of this phase, you will have :

- Incident response procedures
- Troubleshooting experience
- Recovery procedures
- Operational documentation
- Professional incident reports

II. Incident Response Methodology

Use the same methodology for every incident.

1. Detection
2. Impact Assessment
3. Investigation
4. Root Cause Identification
5. Resolution
6. Validation
7. Documentation

1. Incident 1 – PostgreSQL Service Failure

i. Scenario

The PostgreSQL service unexpectedly stops.

ii. Simulation

On pg-primary :

```
sudo systemctl stop postgresql
```

iii. Detection

Run :

```
systemctl status postgresql
```

or :

```
./check_database.sh
```

Prometheus should show :

```
postgresql exporter DOWN
```

iv. Investigation

Check logs :

```
sudo journalctl -u postgresql
```

Check service :

```
sudo systemctl status postgresql
```

Verify port :

```
ss -tlnp | grep 5432
```

v. Resolution

```
sudo systemctl start postgresql
```

Verify :

```
sudo systemctl status postgresql
```

vi. Validation

```
psql -h pg-primary \  
-U app_user \  
-d company_prod
```

vii. RCA Example

- Root Cause
Service manually stopped during simulation.
- Resolution
PostgreSQL service restarted.
- Preventive Action
Enable service monitoring alerts.

2. Incident 2 – Disk Space Exhaustion

i. Scenario

Database server runs out of disk space.

ii. Simulation

Create large files :

```
dd if=/dev/zero \  
of=/tmp/filldisk.bin \  
bs=1M \  
count=5000
```

Adjust based on available storage.

iii. Detection

```
df -h
```

Gafana :

Disk usage spike

iv. Investigation

Largest files :

```
du -ah / | sort -rh | head -20
```

Database size :

```
SELECT
pg_size_pretty(
pg_database_size('company_prod')
);
```

v. Resolution

Delete file :

```
rm -f /tmp/filldisk.bin
```

Verify :

```
df -h
```

vi. RCA Example

- Root Cause
Storage consumed by temporary file.
- Resolution
Removed unnecessary data.
- Preventive Action
Disk capacity monitoring.

3. Incident 3 – Replication Failure

i. Scenario

Replica loses synchronization.

ii. Simulation

On pg-replica :

```
sudo systemctl stop postgresql
```

iii. Detection

On primary :

```
SELECT *
FROM pg_stat_replication;
```

Prometheus :

Replica unavailable

iv. Investigation

Replica service :

```
systemctl status postgresql
```

Logs :

```
journalctl -u postgresql
```

v. Resolution

```
sudo systemctl start postgresql
```

Verify :

```
SELECT
state
FROM pg_stat_replication;
```

Expected :

streaming

vi. Validation

- Insert test record.
- Verify on replica.

4. Incident 4 – Backup Failure

i. Scenario

Backup job stops working.

ii. Simulation

Rename backup script :

```
mv postgres_logical_backup.sh \
postgres_logical_backup.old
```

iii. Detection

```
ls /backups/postgresql/logical
```

Cron logs :

```
grep CRON /var/log/syslog
```

iv. Investigation

```
crontab -l
```

Check script path.

v. Resolution

```
mv postgres_logical_backup.old \
postgres_logical_backup.sh
```

Run manually :

```
./postgres_logical_backup.sh
```

vi. Validation

Verify new backup exists.

5. Incident 5 – Authentication Failure**i. Scenario**

Application user cannot connect.

ii. Simulation

Change password :

```
ALTER ROLE app_user
PASSWORD 'WrongPassword';
```

iii. Detection

```
./check_database.sh
```

Expected :

authentication failed

iv. Investigation

Check logs :

```
tail -f \
/var/log/postgresql/postgresql-*.log
```

Expected:

password authentication failed

v. Resolution

Restore correct password :

```
ALTER ROLE app_user
PASSWORD 'CorrectPassword';
```

vi. Validation

Successful connection.

6. Incident 6 – High CPU Utilization**i. Scenario**

A poorly optimized query consumes CPU.

ii. Simulation

Execute :

```
SELECT *
FROM generate_series(1,50000000);
```

iii. Detection

- Grafana dashboard
 - CPU spike

- Linux
 - top
- Database

```
SELECT *
FROM pg_stat_activity;
```

iv. Investigation

Identify query :

```
SELECT
pid,
query,
state
FROM pg_stat_activity;
```

v. Resolution

Terminate query :

```
SELECT pg_terminate_backend(pid);
```

Use identified PID.

vi. Validation

CPU returns to normal.

7. Incident 7 – Long Running Transaction

i. Scenario

Transaction remains open indefinitely.

ii. Simulation

Session 1 :

```
BEGIN;
UPDATE hr.employees
SET department='IT';
```

Do not commit.

iii. Detection

Session 2 :

```
SELECT
pid,
state,
query
FROM pg_stat_activity;
```

iv. Investigation

Find :

```
SELECT
xact_start,
query
FROM pg_stat_activity;
```

v. Resolution

Terminate :

```
SELECT pg_terminate_backend(pid);
```

vi. Validation

Locks released.

8. Incident 8 – Unauthorized Access Attempt**i. Scenario**

Attacker attempts database login.

ii. Simulation

```
psql \
-U hacker \
-d company_prod
```

iii. Detection

Logs :

```
tail -f \
/var/log/postgresql/postgresql-*.log
```

Expected :

authentication failed

iv. Investigation

Check :

```
grep authentication \
/var/log/postgresql/*.log
```

v. Resolution

Verify :

- Firewall
- pg_hba.conf
- User permissions

vi. Validation

Unauthorized login blocked.

9. Incident 9 – Monitoring Failure

i. Scenario

Prometheus stops collecting metrics.

ii. Simulation

On monitoring :

```
sudo systemctl stop prometheus
```

iii. Detection

- Grafana
 - No data

Verify :

```
systemctl status prometheus
```

Expected :

inactive

iv. Resolution

```
sudo systemctl start prometheus
```

v. Validation

Targets return :

UP

10. Incident 10 – Complete Primary Server Failure

i. Scenario

Primary server becomes unavailable.

ii. Simulation

Stop PostgreSQL :

```
sudo systemctl stop postgresql
```

or power off VM.

iii. Detection

- Connection timeout
- Prometheus alert :
 - Database unavailable

iv. Investigation

Verify :

```
ping pg-primary
```

```
ssh pg-primary
```

v. Resolution

Promote standby

On pg-replica :

```
SELECT pg_promote();
```

Verify :

```
SELECT pg_is_in_recovery();
```

Expected :

false

vi. Validation

Application connects to promoted replica.

III. Root Cause Analysis Template

Create :

```
operations/runbooks/incident_template.md
```

```
# Incident Report
```

```
## Incident ID
```

```
INC-001
```

```
## Date
```

```
YYYY-MM-DD
```

```
## Summary
```

```
Brief description
```

```
## Impact
```

```
Services affected
```

```
## Detection Method
```

```
Monitoring / User Report
```

```
## Root Cause
```

```
Technical cause
```

```
## Resolution
```

```
Actions performed
```

```
## Validation
```

```
Verification steps
```

```
## Preventive Actions
```

```
Future improvements
```